

计算几何

目录 I

1 基础内容

2 二维凸包

目录

1 基础内容

2 二维凸包

图形

点：我们一般使用坐标维护，即使用一个有序二元组存坐标。

图形

点：我们一般使用坐标维护，即使用一个有序二元组存坐标。
向量：与点的表示方法完全一致。



图形

点：我们一般使用坐标维护，即使用一个有序二元组存坐标。

向量：与点的表示方法完全一致。

线：线的解析式一般式为 $Ax + By + C = 0$ ，但是这个式子不一定方便好用。所以我们还有初中常用的斜截式 $y = kx + b$ ，以及截距式 $\frac{x}{a} + \frac{y}{b} = 1$ 。



公式

解三角形常用的公式，在任意三角形 $\triangle ABC$ 中，若三角对应边分别为 a, b, c ，有：



公式

解三角形常用的公式，在任意三角形 $\triangle ABC$ 中，若三角对应边分别为 a, b, c ，有：

正弦定理： $\frac{a}{\sin A} = 2R$ ，其中 R 为三角形外接圆半径。



公式

解三角形常用的公式，在任意三角形 $\triangle ABC$ 中，若三角对应边分别为 a, b, c ，有：

正弦定理： $\frac{a}{\sin A} = 2R$ ，其中 R 为三角形外接圆半径。

余弦定理： $a^2 = b^2 + c^2 - 2bc \cos A$ 。



公式

解三角形常用的公式，在任意三角形 $\triangle ABC$ 中，若三角对应边分别为 a, b, c ，有：

正弦定理： $\frac{a}{\sin A} = 2R$ ，其中 R 为三角形外接圆半径。

余弦定理： $a^2 = b^2 + c^2 - 2bc \cos A$ 。

设一个多边形所有点按照逆时针排列为 p 数组，设任意点 O ，向量 $v_i = p_i - O$ ，则多边形面积

$$S = \frac{1}{2} \left| \sum_{i=1}^n v_i \times v_{i \bmod n+1} \right|。$$



公式

解三角形常用的公式，在任意三角形 $\triangle ABC$ 中，若三角对应边分别为 a, b, c ，有：

正弦定理： $\frac{a}{\sin A} = 2R$ ，其中 R 为三角形外接圆半径。

余弦定理： $a^2 = b^2 + c^2 - 2bc \cos A$ 。

设一个多边形所有点按照逆时针排列为 p 数组，设任意点 O ，向量 $v_i = p_i - O$ ，则多边形面积

$$S = \frac{1}{2} \left| \sum_{i=1}^n v_i \times v_{i \bmod n+1} \right|。$$

如何判断一个点是否在一个凸多边形中？



极角排序

我们希望把所有点按照其极角来排序，正常实现这一点会比较困难，因为我们会试图求出 $\tan x$ 的反函数，利用斜率来比较，然而 $\arctan$ 的值域只有 $(-\frac{\pi}{2}, \frac{\pi}{2})$ 。且垂直的时候斜率不存在。



极角排序

我们希望把所有点按照其极角来排序，正常实现这一点会比较困难，因为我们会试图求出 $\tan x$ 的反函数，利用斜率来比较，然而 $\arctan$ 的值域只有 $(-\frac{\pi}{2}, \frac{\pi}{2})$ 。且垂直的时候斜率不存在。于是可以使用 C++ 自带的 $\text{atan2}(y, x)$ 来进行这个操作，这样就方便比较了。



极角排序

我们希望把所有点按照其极角来排序，正常实现这一点会比较困难，因为我们会试图求出 $\tan x$ 的反函数，利用斜率来比较，然而 $\arctan$ 的值域只有 $(-\frac{\pi}{2}, \frac{\pi}{2})$ 。且垂直的时候斜率不存在。于是可以使用 C++ 自带的 $\text{atan2}(y, x)$ 来进行这个操作，这样就方便比较了。

例题：P14428。

欧几里得距离

在二维平面上，有勾股定理，得两点距离为 $\sqrt{\Delta x^2 + \Delta y^2}$ 。

欧几里得距离

在二维平面上，有勾股定理，得两点距离为 $\sqrt{\Delta x^2 + \Delta y^2}$ 。

其实也容易拓展到 n 维空间中，可以得到欧式距离的一般式 $\sqrt{\sum_{i=1}^n \Delta x_i^2}$ 。

欧几里得距离

在二维平面上，有勾股定理，得两点距离为 $\sqrt{\Delta x^2 + \Delta y^2}$ 。

其实也容易拓展到 n 维空间中，可以得到欧式距离的一般式 $\sqrt{\sum_{i=1}^n \Delta x_i^2}$ 。

例题：P3958。

曼哈顿距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的曼哈顿距离为 $|x_1 - x_2| + |y_1 - y_2|$ 。也就是只允许在坐标轴方向上行走的距离。

曼哈顿距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的曼哈顿距离为 $|x_1 - x_2| + |y_1 - y_2|$ 。也就是只允许在坐标轴方向上行走的距离。

容易发现一个性质：曼哈顿距离下的圆是一个旋转了 $\frac{\pi}{4}$ 的正方形。

曼哈顿距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的曼哈顿距离为 $|x_1 - x_2| + |y_1 - y_2|$ 。也就是只允许在坐标轴方向上行走的距离。

容易发现一个性质：曼哈顿距离下的圆是一个旋转了 $\frac{\pi}{4}$ 的正方形。

例题：P5098。

曼哈顿距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的曼哈顿距离为 $|x_1 - x_2| + |y_1 - y_2|$ 。也就是只允许在坐标轴方向上行走的距离。

容易发现一个性质：曼哈顿距离下的圆是一个旋转了 $\frac{\pi}{4}$ 的正方形。

例题：P5098。

于是我们会发现，两个点的曼哈顿距离其实就是 $\max(|x_1 + y_1 - x_2 - y_2|, |x_1 - y_1 - (x_2 - y_2)|)$ 。

切比雪夫距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的切比雪夫距离为 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 。

切比雪夫距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的切比雪夫距离为 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 。
容易发现一个性质：切比雪夫距离下的圆是一个正方形。

切比雪夫距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的切比雪夫距离为 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 。

容易发现一个性质：切比雪夫距离下的圆是一个正方形。

而对比曼哈顿距离 $|x_1 - x_2| + |y_1 - y_2| = \max(|x_1 + y_1 - x_2 - y_2|, |x_1 - y_1 - (x_2 - y_2)|)$ ，其实就是点 $(x_1 + y_1, x_1 - y_1)$ 与点 $(x_2 + y_2, x_2 - y_2)$ 的切比雪夫距离。

切比雪夫距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的切比雪夫距离为 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 。

容易发现一个性质：切比雪夫距离下的圆是一个正方形。

而对比曼哈顿距离 $|x_1 - x_2| + |y_1 - y_2| = \max(|x_1 + y_1 - x_2 - y_2|, |x_1 - y_1 - (x_2 - y_2)|)$ ，其实就是点 $(x_1 + y_1, x_1 - y_1)$ 与点 $(x_2 + y_2, x_2 - y_2)$ 的切比雪夫距离。

回顾转轴公式，这恰好是把点 (x, y) 旋转了 $\frac{\pi}{4}$ ，再缩放 $\sqrt{2}$ 倍的结果。这样一来也刚好说得通了。

切比雪夫距离

定义两点 $A(x_1, y_1), B(x_2, y_2)$ 的切比雪夫距离为 $\max(|x_1 - x_2|, |y_1 - y_2|)$ 。

容易发现一个性质：切比雪夫距离下的圆是一个正方形。

而对比曼哈顿距离 $|x_1 - x_2| + |y_1 - y_2| = \max(|x_1 + y_1 - x_2 - y_2|, |x_1 - y_1 - (x_2 - y_2)|)$ ，其实就是点 $(x_1 + y_1, x_1 - y_1)$ 与点 $(x_2 + y_2, x_2 - y_2)$ 的切比雪夫距离。

回顾转轴公式，这恰好是把点 (x, y) 旋转了 $\frac{\pi}{4}$ ，再缩放 $\sqrt{2}$ 倍的结果。这样一来也刚好说得通了。因此得到重要结论：切比雪夫距离和曼哈顿距离等价，二者可以相互转化。

例题

qoj9902, P3439, P4648, P3964

目录

1 基础内容

2 二维凸包

介绍

凸多边形就是所有内角都在 $[0, \pi]$ 的简单多边形。

介绍

凸多边形就是所有内角都在 $[0, \pi]$ 的简单多边形。

凸包的定义就是：给出平面上的一些点，能把所有点包括的最小凸多边形就是凸包。

介绍

凸多边形就是所有内角都在 $[0, \pi]$ 的简单多边形。

凸包的定义就是：给出平面上的一些点，能把所有点包括的最小凸多边形就是凸包。

你也可以理解为，一些点的凸包就是平面上在这些点的位置插钉子，用一根最短的橡皮筋套住这些点的图像就是这些点的凸包。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

Andrew 算法就是分别求出上下凸包从而合并出整个凸包的算法。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

Andrew 算法就是分别求出上下凸包从而合并出整个凸包的算法。

以求下凸包为例，我们按照横坐标排序，动态加入一个节点，维护当前的凸包。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

Andrew 算法就是分别求出上下凸包从而合并出整个凸包的算法。

以求下凸包为例，我们按照横坐标排序，动态加入一个节点，维护当前的凸包。

我们维护一个单调栈，存贮目前的凸包中的所有点（按照横坐标排序）。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

Andrew 算法就是分别求出上下凸包从而合并出整个凸包的算法。

以求下凸包为例，我们按照横坐标排序，动态加入一个节点，维护当前的凸包。

我们维护一个单调栈，存贮目前的凸包中的所有点（按照横坐标排序）。

当我们试图加入一个点 P 时，现在栈顶的两个点分别为 S_1, S_2 ，其中 S_1 为栈顶。考虑向量 $\vec{v_1} = \vec{S_2S_1}$ 与 $\vec{v_2} = \vec{S_2P}$ 如果 $\vec{v_2}$ 在 $\vec{v_1}$ 的顺时针方向，则弹出 S_1 ，不断执行这个操作直到不能弹出或者栈内元素不足。最后将 P 加入凸包即可。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

Andrew 算法就是分别求出上下凸包从而合并出整个凸包的算法。

以求下凸包为例，我们按照横坐标排序，动态加入一个节点，维护当前的凸包。

我们维护一个单调栈，存贮目前的凸包中的所有点（按照横坐标排序）。

当我们试图加入一个点 P 时，现在栈顶的两个点分别为 S_1, S_2 ，其中 S_1 为栈顶。考虑向量 $\vec{v_1} = \vec{S_2, S_1}$ 与 $\vec{v_2} = \vec{S_2, P}$ 如果 $\vec{v_2}$ 在 $\vec{v_1}$ 的顺时针方向，则弹出 S_1 ，不断执行这个操作直到不能弹出或者栈内元素不足。最后将 P 加入凸包即可。

判断两个向量顺时针方向也是很简单的东西，只需要叉乘看正负即可。

求凸包

首先注意到一个性质：把所有点按照横坐标排序，那么最左与最右的点一定在凸包里面。而其他的点可以被就此分为上凸包和下凸包。

Andrew 算法就是分别求出上下凸包从而合并出整个凸包的算法。

以求下凸包为例，我们按照横坐标排序，动态加入一个节点，维护当前的凸包。

我们维护一个单调栈，存贮目前的凸包中的所有点（按照横坐标排序）。

当我们试图加入一个点 P 时，现在栈顶的两个点分别为 S_1, S_2 ，其中 S_1 为栈顶。考虑向量 $\vec{v_1} = \vec{S_2, S_1}$ 与 $\vec{v_2} = \vec{S_2, P}$ 如果 $\vec{v_2}$ 在 $\vec{v_1}$ 的顺时针方向，则弹出 S_1 ，不断执行这个操作直到不能弹出或者栈内元素不足。最后将 P 加入凸包即可。

判断两个向量顺时针方向也是很简单的东西，只需要叉乘看正负即可。

复杂度的瓶颈在于排序。

闵可夫斯基和

定义点集 P, Q 的闵可夫斯基和为点集 $P + Q = \{a + b | a \in P, b \in Q\}$ 。

闵可夫斯基和

定义点集 P, Q 的闵可夫斯基和为点集 $P + Q = \{a + b \mid a \in P, b \in Q\}$ 。
而我们会经常试图在已知 P, Q 的凸包时，求出 $P + Q$ 的凸包。

闵可夫斯基和

定义点集 P, Q 的闵可夫斯基和为点集 $P + Q = \{a + b | a \in P, b \in Q\}$ 。

而我们会经常试图在已知 P, Q 的凸包时，求出 $P + Q$ 的凸包。

结论是： $P + Q$ 凸包的所有边就是 P, Q 凸包中所有边的合并后的边。所以只用合并之后再对边做极角排序即可。

例题

P2742, P8101, P3829, P4557

旋转卡壳

这四个字到底怎么读？

旋转卡壳

这四个字到底怎么读？

其实这东西也不是什么算法，而只能算是一种处理凸包问题的套路，也就是在凸包上做双指针。

旋转卡壳

这四个字到底怎么读？

其实这东西也不是什么算法，而只能算是一种处理凸包问题的套路，也就是在凸包上做双指针。

直接给出例题：P1452, P3187